

Laboratorio 5

Regresión lineal y Clasificación

Valentina Pérez Cadavid

2026-2

Parte 1

Ejercicio 1.1

Recordar que se estableció como correcto $\theta = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix}$

$$\rightarrow h_{\theta}(x_i^{(i)}) = \theta_0 + \theta_1 x_i^{(i)}$$

$\downarrow$
0.2

300 valores candidatos para θ_1 :

Puede ser: $\theta_{1\text{grid}} = \{-1, \dots, -0.5, \dots, 0, \dots, 0.5, \dots, 1.5, \dots, 2\}$

Intervalo que se grafica como *zimple*

$$\left. \begin{aligned} J_1 &= \frac{1}{2m} \sum_{i=1}^{100} [(\theta_0 + (-1)x_i^{(i)}) - y_i^{(i)}]^2 \\ J_2 &= \frac{1}{2m} \sum_{i=1}^{100} [(\theta_0 + (-0.9)x_i^{(i)}) - y_i^{(i)}]^2 \\ &\vdots \\ J_{300} &= \frac{1}{2m} \sum_{i=1}^{100} [(\theta_0 + 2x_i^{(i)}) - y_i^{(i)}]^2 \end{aligned} \right\} \theta_1 = 0.5 \text{ minimiza } J.$$

La superficie de coste: Con 2 parámetros θ_1, θ_2 , J es una superficie en $\mathbb{R}^3$

Hasta ahora J_{grid} dependía de un solo parámetro θ_1 (con θ_0 fijo, baido en una dimensión).

Pero la superficie de coste real depende de 2 parámetros θ_0 y θ_1 .

Se necesita hacer un barrido sobre ambos. Se debe evaluar J en todas las combinaciones posibles de θ_0 y θ_1 .

→ `np.linspace(-0.6, 1.0, 80)` : Genera 80 candidatos de θ_0

→ `np.linspace(-0.6, 1.0, 80)` : Genera 80 candidatos de θ_1

$80 \times 80 = 6.400$ combinaciones

→ `meshgrid` : devuelve 2 arreglos a la vez

→ Recorrer las 6400 combinaciones (θ_0, θ_1) que armé meshgrid
calcula J para cada una y guarda todos esos 6,400
resultados en Jg .

$$Jg = \begin{bmatrix} Jg(1,1) & \dots & Jg(1,80) \\ Jg(2,1) & & \vdots \\ \vdots & & \vdots \end{bmatrix}_{80 \times 80}$$

Parte 2: Gradiente Descendente

Para bajar por la superficie se usa la regla de actualización

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

Caso 1D:

Ensayar con función mucho más simple: Una variable

$$f(w) = (w-1)^2 \quad ; \quad f_{\min} \text{ en } w=1$$
$$f'(w) = 2(w-1)$$

Aplicar regla $w := w - \alpha f'(w)$ → w valores que hacen que
 $h_0(w)$ (en este caso) se
ajuste mejor a los datos reales.
minimizando J

- α pequeño: converge, pero gusta muchas iteraciones
- α adecuado: baja rápido y se queda en el mín.
- α muy grande: sobrepasa el mínimo en cada paso y diverge.

Ejercicio 2.1: Implementar el gradiente y el descenso

En este caso, se debe calcular:

$$\theta_0 := \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 := \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

Parte 3: Regresión Multivariada

versión vectorizada

Ya no es una única característica, si no n .

De este modo

$$X = \begin{bmatrix} x_1^{(1)} \\ x_1^{(2)} \\ \vdots \\ x_1^{(m)} \end{bmatrix} \rightarrow X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & & x_n^{(2)} \\ \vdots & \vdots & & \vdots \\ 1 & x_1^{(m)} & & x_n^{(m)} \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

$$h = X\theta \quad ; \quad J(\theta) = \frac{1}{2m} (h-y)^T (h-y) \quad ; \quad \nabla_{\theta} J = \frac{1}{m} X^T (h-y)$$

$$\nabla_{\theta} J = \frac{1}{m} X^T (X\theta - y)$$

• def agregar_sesgo: X_f : es un array

if $X_f.ndim = 1$: $X_f = X_f.reshape(-1, 1)$

Saber cuantas dimensiones tiene el array.

Si tiene una sola característica $\rightarrow$ lo guarda como $[\#, \#, \dots \#]$
 $\text{ndim} = 1$.

`reshape(-1, 1)`

$\downarrow$ np calcula las filas necesarias

$\rightarrow$ Número de columnas

Sin importar si se le da 1 ó n características, X_f siempre termina siendo una matriz y no un vector plano.

• `def h_lin:` $X_b @ \text{Theta}$ multiplicación matriz vector

$\downarrow$
si $X_b = (4 \times 4)$ 4 parámetros
4 células, 4 columnas $\theta_0, \theta_1, \theta_2, \theta_3$

$$h_0(x) = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}_{2 \times 2} \begin{bmatrix} 1 \\ 0 \end{bmatrix}_{2 \times 1} = \begin{bmatrix} 1+0 \\ 3+0 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}_{2 \times 1} = \begin{bmatrix} h_0(x^{(1)}) \\ h_0(x^{(2)}) \end{bmatrix}$$

En general :

$$h_0(x) = \begin{bmatrix} h_0(x^{(1)}) \\ \vdots \\ h_0(x^{(m)}) \end{bmatrix}$$

• `J_lin:` Calcula vector de residuos

$$J_{\text{lin}} = \begin{bmatrix} J_1 \\ \vdots \\ J_m \end{bmatrix}$$

Ejercicio 3.1: El gradiente vectorizado

Traducir $\nabla_{\theta} J = \frac{1}{m} X^T (X\theta - y)$ a una sola línea de numpy.

$$= \frac{1}{m} X.T (h_{\theta}(x) - y) \quad X_b \equiv X$$

$$\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j} \rightarrow \text{Descenso vectorial}$$

• `def_descenso_vect`: Al igual que en el caso de 1 característica

`Theta = None` $\rightarrow$ se pone `None` porque no sabemos cuantos parámetros va a tener θ . Depende del # de características.
Esto añade un valor vacío

`Theta = np.zeros(Xb.shape[1])`

devuelve (filas, columnas)

devuelve el # de columnas

Si `Theta` es `None`, usa un vector de ceros del tamaño relacionado con el # de columnas de `X`, sino, convierte la entrada de `Theta` en un arreglo de # flotantes.

- El gradiente computado, se puede comparar con la definición de derivada. Para esto se crea una función que recibe:
- *función* (en este caso es J_{lm} que solo depende de θ)
 - *Theta* (donde se evalúa el gradiente)
 - ϵ (Tamaño de paso)

→ Se irá guardando cada derivada en g según el # de componentes que tenga θ . $g = \text{np.zeros_like}(\theta)$

→ $\text{for } k \text{ in range}(\text{len}(\theta))$: el gradiente tiene una derivada parcial por cada componente de θ . Este bucle calcula las derivadas una por una.

→ $e = \text{np.zeros_like}(\theta)$ y $e[k] = \epsilon$: construye un vector de ceros excepto en la posición k , donde pone ϵ . Es la manera de medir el efecto de mover solo una variable $\equiv \frac{\partial J}{\partial \theta_k}$.

$$f'(w) \approx \frac{f(w + \epsilon) - f(w - \epsilon)}{2\epsilon}$$

$$J_{lm} \text{ con } \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

$$k = 0, 1$$

$$\hookrightarrow k=0 : \epsilon = \begin{bmatrix} \epsilon \\ 0 \end{bmatrix} \mapsto \theta + \epsilon = \begin{bmatrix} \theta_0 + \epsilon \\ \theta_1 \end{bmatrix}$$

$$\theta - \epsilon = \begin{bmatrix} \theta_0 - \epsilon \\ \theta_1 \end{bmatrix}$$

$$g[0] = \frac{J(\theta_0 + \epsilon, \theta_1) - J(\theta_0 - \epsilon, \theta_1)}{2\epsilon} \equiv \frac{\partial J}{\partial \theta_0}$$



$$\mapsto k=1 : \rightarrow \varepsilon = \begin{bmatrix} 0 \\ \varepsilon \end{bmatrix} \mapsto \theta + \varepsilon = \begin{bmatrix} \theta_0 \\ \theta_1 + \varepsilon \end{bmatrix}$$

$$\theta - \varepsilon = \begin{bmatrix} \theta_0 \\ \theta_1 - \varepsilon \end{bmatrix}$$

$$g[1] = \frac{J(\theta_0, \theta_1 + \varepsilon) - J(\theta_0, \theta_1 - \varepsilon)}{2\varepsilon} \equiv \frac{\partial J}{\partial \theta_1} \quad \checkmark$$

Al final : $g = (g[0], g[1]) = \nabla_{\theta} J(\theta)$

→ `xb_test = agregar_sesgo(rng.normal(size=(30,3)))`

genera 30 mediciones de 3 características con ruido gaussiano
Es decir, le damos cualquier matriz X y cualquier vector y del tamaño necesario.

Al evaluarla en la función `agregar sesgo` le añade la columna de 1.

$$X = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_3^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & x_3^{(2)} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(30)} & x_2^{(30)} & x_3^{(30)} \end{bmatrix}$$

30 x 4

→ `assert np.allclose(g_ana, g_num, atol=1e-6)`

compara los vectores elemento por elemento y devuelve un

True: si la diferencia de los valores es menor o igual al umbral $10^{-6} - 10^{-5}$

False: si la diferencia es mayor. → Imprime el texto "g-ana no coincide con el numérico"

Ejemplo : plano $y = -1 + 0.2x_1 + 0.5x_2$

200 = Número de datos $m = 200$

2 = Número de características $x_1^{(i)}, x_2^{(i)}$ con valores entre -1 y 1

y_2 = Datos generados con la fórmula del plano.

→ `np.column_stack([x1, x2])` : toma los 2 vectores 1D y los pega como columnas en la matriz

→ `Thet_gd, hist2 = descenso_vect(xb, y2, alpha=0.3, n_iter=5000)`

Devuelve el arreglo de Thetas optimos. En este caso es de tamaño 1×3

→ `g1, g2 = np.meshgrid(np.linspace(-1, 1, 20), np.linspace(-1, 1, 20))`

Son 20 valores elegidos aleatoriamente para cada característica a los cuales el modelo les devuelve la altura predicha.

Parte 4: Escalamiento de características

¿Qué pasa si una característica vive en $[-1, 1]$ y otra en $[-2000, 2000]$?

La superficie de coste se vuelve un valle largo y estrecho el mismo α es demasiado grande en una dirección y demasiado pequeño en la otra.

La solución es estandarizar:

$$x_j \leftarrow \frac{x_j - \mu_j}{s_j}$$

μ_j : media
 s_j : desv. estándar

→ with `np.errstate(over="ignore", invalid="ignore")`:

Silenciar las advertencias de tipo overflow (desbordamiento numérico) e invalid (operaciones inválidas como $-\infty, \infty$), que se produce al calcular el coste.

Numpy no imprime el aviso (ya se sabe que diverge)

`hist_mal[:5] = [6.8, 8.7, 1.1, 1.4, 1.8]` ← no divergen

`hist_mal[-1] = [inf]` ← diverge.

Esto sucede tanto para $\alpha = 0.3$ (grande) como para $\alpha = 1e^{-8}$ (pequeño)

Se debe estandarizar y volver a entrenar

Ejercicio 4.1: Estandarizar y volver a entrenar

Recordar que X_f es $(m \times n)$

Recordar que x_f es $(m \times n)$

$x_f \begin{bmatrix} & & & \end{bmatrix} \begin{bmatrix} & & & \end{bmatrix}$
 $\downarrow \quad \downarrow$
 fila columna

$$X = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_n^{(1)} \\ 1 & x_1^{(2)} & & x_n^{(2)} \\ \vdots & \vdots & & \vdots \\ 1 & \underbrace{x_1^{(m)}}_{\sigma, \mu} & & \underbrace{x_n^{(m)}}_{\sigma, \mu} \end{bmatrix} = \begin{bmatrix} [0][0] & [0][1] & [0][2] & \dots & [0][n] \\ [1][0] & & & & \vdots \\ [2][0] & & & & \\ \vdots & & & & \\ [m][0] & & & & [m][n] \end{bmatrix}$$

→ assert np.allclose(z_esc.mean(axis=0), 0, atol=1e-10)

Tomar z-esc y calcular el promedio de cada columna por separado

Compara cada valor de la media de la columna con 0 y devuelve:

True: solo si ambos están a $1e^{-10}$ de diferencia de 0

Cuando los datos han sido estandarizados, calcular la media sobre ellos da 0.

Los datos estandarizados, el α puede usarse para ambas características. Sin embargo el modelo resultante θ_{esc} queda con una representación diferente. En lugar de decir la relación entre la característica y la variable que se quiere predecir, da una relación entre la desviación estándar de la característica y la variable que se quiere predecir.

El escalamiento es una herramienta interna para entrenar mejor. Cuando el modelo ya aprendió no se necesita usar esta transformación.

$$\theta_j = \frac{\theta_{j, \text{esc}}}{\sigma_j} \quad j = 1, 2, \dots, n$$
$$\theta_0 = \theta_{0, \text{esc}} - \sum_{j=1}^n \frac{\theta_{j, \text{esc}} \cdot \mu_j}{\sigma_j}$$

Parte 5: La ecuación normal

$$\underbrace{X^T X}_A \theta = \underbrace{X^T y}_b \longrightarrow \theta = (X^T X)^{-1} X^T y$$

np.linalg.solve(A, b) resuelve $A\theta = b$.

Comparando los θ por:

θ Ecuación normal = θ descenso gradiente

se llega al mismo mínimo

Ejemplo: Datos reales diabetes

$m = 442$ datos

$n = 10$ características: edad, presión, sangre, ...

y_d = medida de progresión de diabetes (a estimar)

→ `skl = LinearRegression().fit(xd, yd)`

Como no hay un θ_{real} para comparar, se usa skl que resuelve con mínimos cuadrados

Gradiente Descendente o Ecuación Normal?

La ecuación normal solo existe porque $J(\theta)$ es cuadrática en regresión lineal

En regresión logística J ya no es cuadrática por la sigmoide

Parte 6: Clasificación Regresión Logística

Ahora $y \in \{0, 1\}$.

Queremos que la salida sea una probabilidad, así que pasamos $\theta^T x$ por la sigmoide

$$h_\theta(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}} = P(y=1 | x; \theta) \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La regla de decisión es simplemente el signo del argumento

$$\theta^T x \geq 0 \rightarrow h \geq 0.5 \rightarrow \text{Clase 1}$$

$$\theta^T x < 0 \rightarrow h < 0.5 \rightarrow \text{Clase 0}$$

La ecuación $\theta^T x = 0$ define la frontera de decisión
con 2 características (x_1, x_2) la frontera es una línea
con mas de 2 la frontera es un hiperplano.

Ejercicio 6.1 Encuentre la frontera circular

Nada obliga a que $\theta^T x$ sea lineal en las características originales
Si agregamos términos polinómicos, por ejemplo

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 \geq 0$$

La frontera puede ser una circunferencia, una elipse ...

Con $\theta = [-r^2, 0, 0, 1, 1]$

$$\theta^T x = -r^2 + 0 \cdot x_1 + 0 \cdot x_2 + 1 \cdot x_1^2 + 1 \cdot x_2^2 = x_1^2 + x_2^2 - r^2$$

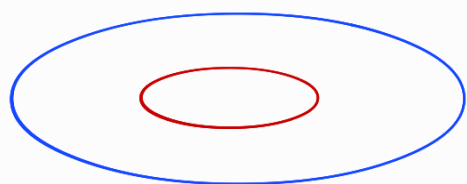
lo cual según la regla de decisión:

$$\theta^T x = 0 = x_1^2 + x_2^2 - r^2$$

$$x_1^2 + x_2^2 = r^2$$

→ `make_circles (n_samples = 300, factor = 0.5, noise = 0.05, random = 0)`

Genera 300 puntos en 2 anillos concéntricos



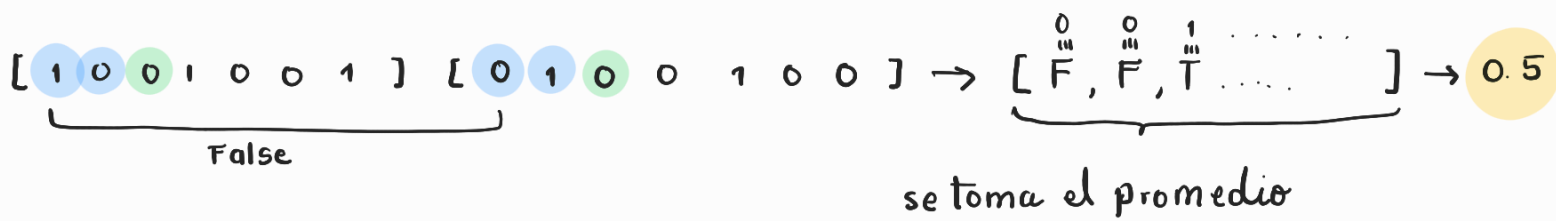
$$x_f = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} \\ x_1^{(2)} & x_2^{(2)} \\ \vdots & \vdots \\ x_1^{(m)} & x_2^{(m)} \end{bmatrix}$$

$$\begin{aligned} x_1^2 + x_2^2 < r^2 &\rightarrow 1 \\ x_1^2 + x_2^2 \geq r^2 &\rightarrow 0 \end{aligned}$$

→ `clasifica_circulo (x_f, r)` : Devuelve un array de 1 y 0 que indica si el punto de esa posición está o no dentro del círculo de radio r.

→ `exactitudes` : por cada r en radios, ejecuta `clasifica_circulo` y los compara con `yr` (arreglo real).

Para cada radio en radios, realiza la comparación :



Devuelve el arreglo de medias $[\dots]$ y encuentra la mas similar.

Ejercicio 6.2: Entrenar un clasificador desde cero

Cuando y se limita a $(0, 1)$, entonces la función de coste es la entropía cruzada dada por:

$$J_{\text{log}} \equiv J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1-y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

$$\nabla_{\theta} J = \frac{1}{m} X^T (h_{\theta}(X) - y) \quad ; \quad h_{\theta}(x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$\theta := \theta - \frac{\alpha}{m} X^T (\sigma(X\theta) - y)$$

→ $\text{pred} = (h_{\text{log}}(\text{Theta}_{\text{log}}, x_{cb}) \geq 0.5). \text{astype(int)}$

$h_{\text{log}}(\text{Theta}_{\text{log}}, x_{cb})$: calcula con el Theta ya entrenado la probabilidad predicha (función sigmoide) para cada una de las medidas

$h_{\text{log}}(\text{Theta}_{\text{log}}, x_{cb}) \geq 0.5$: Comparar el arreglo con 0.5
 Si la comparación es correcta → True
 " " " no " " → False

$[0.03 \quad 0.1 \quad 0.57 \quad 0.61 \dots] = [\text{False}, \text{False}, \text{True}, \text{True} \dots]$

→ `acc_propio = (pred == yc).mean` :

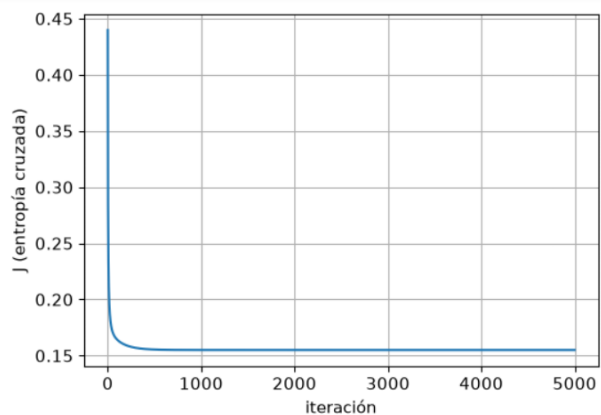
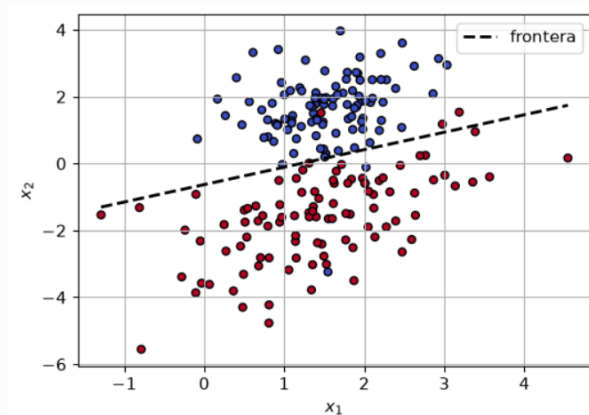
compara los valores predichos con los cargados y_c inicialmente. Esto es un arreglo de True, False.

Se calcula el promedio de ese arreglo. Calcula la fracción de aciertos es decir, la exactitud.

Para visualizar los datos, se parte de la ecuación de la frontera con los datos : x_c, y_c generados por `make_classification`

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$$

$$\theta_2 x_2 = -(\theta_0 + \theta_1 x_1) \rightarrow x_2 = -\frac{\theta_0 + \theta_1 x_1}{\theta_2}$$



Parte 7: Multiclases y Regularización

Hasta ahora se ha hecho una clasificación binaria: 0 ó 1.

Para poder realizar una multclasificación de datos (multiclases) se debe responder la pregunta: # de valores que puede tomar y

¿Cómo se puede extender la sigmoide binaria a $K > 2$ clases?

Hasta ahora, al implementar descenso vectorial ó descenso logístico se midió que tan bien el modelo ajustaba los mismos datos que entrenó. Ahora, se deben separar los datos en:

↙ ↘
Datos para entrenar Datos de prueba

Hay 2 opciones para trabajar con $K > 2$ clases:

1. Se entrenan K clasificadores binarios (clase K vs el resto) y se elige el de mayor probabilidad
2. Softmax: se generaliza la sigmoide a:

$$P(y = k | x) = \frac{e^{\theta_k^T x}}{\sum_j e^{\theta_j^T x}}$$

y se entrena un solo modelo

Se debe recordar que la Regularización, es lo que evita que el modelo "memorice" en lugar de "aprender"

$$J_r(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

$$\theta_j := \theta_j - \frac{\alpha \lambda}{m} \theta_j - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$\lambda = 0 \rightarrow$ modelo libre de ajustarse todo lo que quiera $\rightarrow$ sobreajustable
 $\lambda \gg 0 \rightarrow$ el modelo se vuelve casi constante, no aprende $\rightarrow$ subajustable

$$C = \frac{1}{\lambda}$$

C pequeño $\rightarrow$ mas regularización $\rightarrow$ mas subajustable
 C grande $\rightarrow$ poca regularización $\rightarrow$ modelo mas libre

$\rightarrow$ x_{tr} y_{tr} datos entrenamiento
 x_{te} y_{te} datos prueba

$\rightarrow$ `make_pipeline(StandardScaler(), LogisticRegression(c=1, max_iter=1000))`

Hace la estandarización que se hizo en 4.

$$x \leftarrow \frac{x - \mu}{\sigma}$$

Configuración del modelo

análogo a descenso_log. Mismo

objetivo: minimizar J y encontrar θ

$c=1$ Parámetro de regularización

→ `modelo.fit(xtr, ytr)` : estandariza x_{tr} (calcula μ, σ con datos de entrenamiento y luego entrena la regresión logística sobre esos datos ya escalados)

→ `flor = np.array([[5.9, 3.0, 5.1, 1.8]])`

`np.array([ ])` → vector (1D)

`np.array([ [ ] ])` → matriz de 1 fila (2D)

`flor` representa : 1 flor con 4 mediciones

→ `p = modelo.predict_proba(flor)[0]`

Devuelve una tabla de forma ($m = \# \text{datos}$, $k = \# \text{de clases}$) (1, 3)

Saca la única fila de esa tabla `[0]`. Así que `p` pasa de ser una tabla a un vector plano.

→ `zip(iris.target_names, p)` empareja las secuencias:

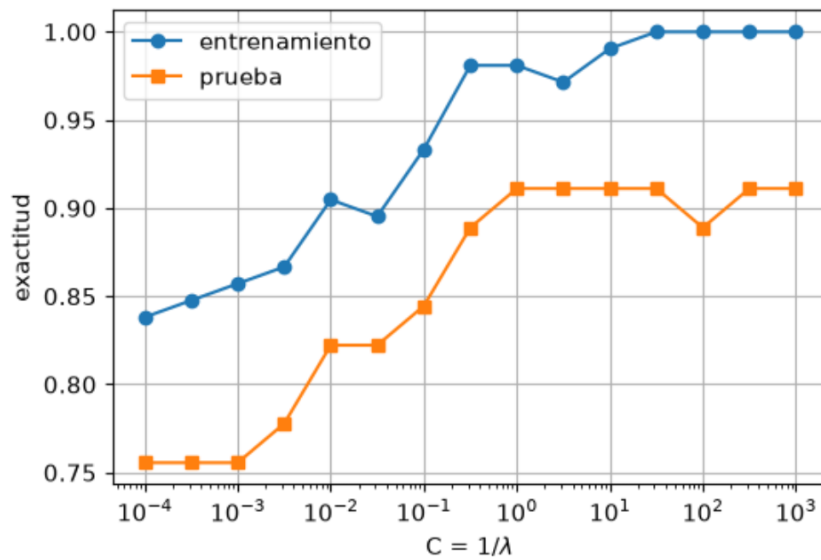
`iris.target_names = [ setosa , versicolor , virginica ]`

`p = [ 0.05 , 0.341 , 0.653 ]`

`[ setosa , versicolor , virginica ]`
`[ 0.05 , 0.341 , 0.653 ]`

Luego se desempaquetan en 2 variables: nombre y prob.

Ejercicio 7.1 El efecto de la Regularización



Se observa la exactitud en función de la regularización

- A mayor regularización (valores pequeños de C), el modelo es bastante restringido.
- Se observa que con los datos de entrenamiento, el modelo se ajusta cada vez mejor a lo que ya vio.
- Con los datos de prueba, se estanca al inicio y nunca llega a 1. Alcanza exactitud máxima en $C=1$. El modelo ya tiene la libertad para aprender el patrón real de las 3 especies sin sobreajustarse.

Para $C > 1$ se ve el sobreajuste. La azul sube pero la naranja se estanca. El modelo gasta su libertad extra memorizando detalles que no generalizan los datos de prueba.

Preguntas de Análisis

1. Por qué el error cuadrático produce una superficie de coste convexa mientras que aplicado a la sigmoide no?

RH Recordar que con la sigmoide : $\sigma(z)$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$J = \frac{1}{m} \sum (\underbrace{\sigma(\theta^T x)} - y)^2$$

Al elevar al cuadrado el término de la sigmoide (curvo), añade curvatura extra.

El error cuadrático medio produce una superficie de coste convexa cuando se aplica regresión lineal porque la relación entre las predicciones y los pesos es lineal.

Por otro lado, el error cuadrático medio no produce una superficie convexa cuando se aplica regresión logística (sigmoide). Dado que la relación entre las predicciones y los pesos no es lineal entonces esa no linealidad transforma la superficie en una función no convexa, con mínimos locales.

2. Usted entrena con $\alpha = 0.01$ durante 100 iteraciones y el coste baja muy poco. Enumere 2 causas posibles y cómo distinguirlas.

R11 causa #1 α muy pequeño

Podría distinguirse si se corren muchas más iteraciones (5000) con el mismo α . Si el coste sigue bajando constantemente y sin oscilar, entonces esto indica que el paso es pequeño y que el algoritmo es lento.

causa #2: características sin escalar

cuando una característica tiene un rango mucho mayor que otra, un α razonable para una de ellas puede no serlo para la otra y el descenso podría darse imperceptiblemente

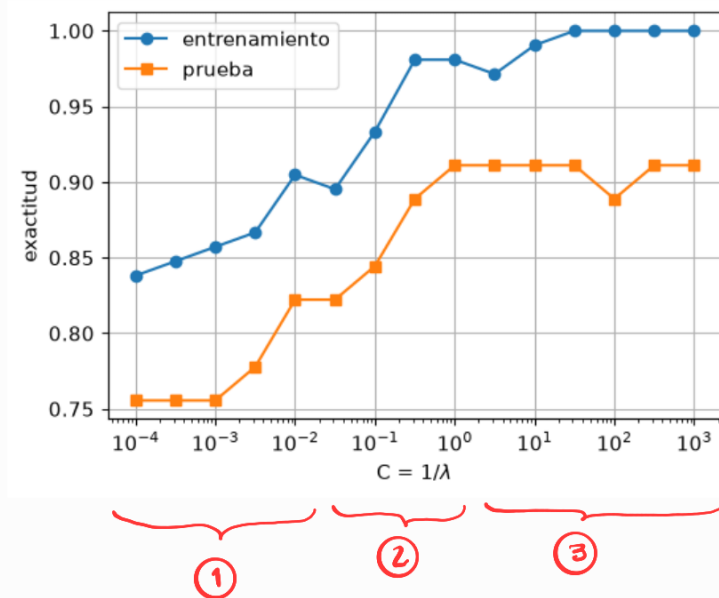
Se debería verificar estandarizando los datos y volver a correr con el mismo α . Si el coste baja mucho más rápido entonces esto lo confirma

3. Con $n = 50.000$ características usaría la ecuación normal?

R11 No. $X^T X \rightarrow (n+1) \times (n+1) = 50.001 \times 50.001$

el costo de invertirla $O(n^3) \approx 1.25 \times 10^{14}$ operaciones

4. En la gráfica del ejercicio 3.1. ¿qué región es de subajuste y de sobreajuste? ¿qué valor de C reportaría?



- ① Con mas regularizacion, el modelo está bastante restringido que no lo gra ajustar bien los datos de entrenamiento. Hay subajuste
- ③ Para $C > 10$ se ve el sobreajuste. la azul sube pero la naranja se estanca. El modelo gasta su libertad extra memorizando detalles que no generalizan los datos de prueba
- ② Es el punto donde la curva de prueba es máxima

$C=1$ valor a reportar